

Universidade Tiradentes
CIÊNCIA DA COMPUTAÇÃO

CAIO VASCONCELOS ARAÚJO FIGUEIREDO
GABRIEL OLIVEIRA EVANGELISTA LUIZ
FELIPE PRUDENTE BORGES

COMPILADORES – UNIDADE II
DESENVOLVIMENTO DO COMPILADOR PIPELANG

Aracaju - SE

2025

CAIO VASCONCELOS ARAÚJO FIGUEIREDO
GABRIEL OLIVEIRA EVANGELISTA LUIZ
FELIPE PRUDENTE BORGES

COMPILADORES – UNIDADE II
DESENVOLVIMENTO DO COMPILADOR PIPELANG

ATIVIDADE apresentada como requisito parcial da disciplina Compiladores,
ministrada pela Prof Layse Souza, no curso de Ciência da Computação, Grupo 3

Aracaju - SE
2025

Sumário

1 INTRODUÇÃO	4
2 JUSTIFICATIVA	4
3 OBJETIVOS	4
3.1 OBJETIVO GERAL	4
3.2 OBJETIVOS ESPECÍFICOS	4
4 METODOLOGIA	5
5 RESULTADOS E DISCUSSOES	5
6 CONSIDERAÇÕES FINAIS	9
7 REFERENCIAS	10
8 ANEXOS	10

1. INTRODUÇÃO

O crescimento do uso de dados nas organizações exige ferramentas capazes de orquestrar fluxos complexos de processamento. Para esse propósito, a disciplina de Compiladores propôs o desenvolvimento de uma DSL (Domain-Specific Language) chamada PipeLang, direcionada para a criação, validação e execução de pipelines ETL/ELT.

Na Unidade I, foram construídos os módulos de análise léxica e sintática. Nesta segunda unidade, o objetivo foi concluir o compilador, adicionando análise semântica, geração de código intermediário, execução e simulação dos pipelines.

O presente documento descreve o processo completo de construção do compilador, detalhando tokens, gramática, semântica, estrutura da IR e o simulador criado.

2. JUSTIFICATIVA

A construção de um compilador completo para uma DSL de pipelines permite que conceitos teóricos de compiladores sejam aplicados em um contexto prático, onde a representação de dependências, tarefas e artefatos representa comportamentos reais de sistemas modernos como Apache Airflow e Luigi. Dessa forma, a atividade promove a compreensão da importância da análise semântica, geração de código intermediário e execução controlada de workflows.

3. OBJETIVOS

3.1. Objetivo Geral

Desenvolver um compilador funcional para a linguagem PipeLang, contemplando análise semântica, geração de código intermediário e simulação de execução.

3.2. Objetivos Específicos

- Implementar uma tabela de símbolos e análise semântica;
- Detectar erros de dependência, artefatos e ciclos;
- Gerar código intermediário em JSON e pseudocódigo;
- Construir um simulador capaz de executar pipelines de forma ordenada;

- Documentar todo o processo conforme orientações da disciplina.

4. METODOLOGIA

O compilador foi desenvolvido em Python, utilizando programação orientada a objetos e modularização. Cada fase do processo foi separada em um arquivo independente:

- **lexer.py** → análise léxica com expressões regulares;
- **parser.py** → geração da AST utilizando descida recursiva;
- **semantic.py** → análise semântica, tabela de símbolos, verificação de dependências, inputs e ciclos;
- **codegen.py** → geração da IR em JSON e pseudocódigo;
- **simulator.py** → execução simulada do pipeline;
- **compiler.py** → integração de todas as fases.

Os exemplos disponibilizados no repositório foram utilizados para validação contínua.

5. RESULTADOS E DISCUSSÕES

A seguir são apresentadas as partes principais do compilador.

5.1 Tokens

A análise léxica identifica elementos como:

- PALAVRAS-CHAVE: **pipeline, task, from, to, map, filter, after, and, or.**
- Identificadores
- Strings
- Números
- Operadores lógicos e aritméticos
- Delimitadores: { } () ; , .

O analisador léxico ignora comentários linha e bloco, conta linhas e colunas e lança

exceções de erro léxico quando necessário.

5.2 Gramática (Sintaxe da Linguagem)

O parser implementa a gramática simplificada:

pipeline → PIPELINE IDENTIFICADOR "{" lista_tarefas "}"

lista_tarefas → tarefa ("," tarefa)*

tarefa → TASK IDENTIFICADOR bloco_tarefa

bloco_tarefa → origem [transformação] saida [after]

origem → FROM STRING

saida → TO STRING

transformação → map | filter

Expressões lógicas suportam combinação com **and**, **or** e uso de parênteses.

5.3 Análise Léxica

Testes demonstraram perfeita identificação de tokens, incluindo:

- suporte a comentários complexos
- reconhecimento de palavras-chave mesmo misturadas com identificadores
- precisão na contagem de posição para mensagens de erro

5.4 Análise Sintática

O parser cria uma AST completa, composta por:

- **Pipeline**
- **Tarefa**
- **Origem**
- **Saida**
- **TransformacaoMap**

- **TransformacaoFilter**
- **ExpressaoComparacao e ExpressaoBinaria**

Erros sintáticos são detectados com linha e coluna exatas.

5.5 Análise Semântica

O módulo semântico faz verificações essenciais:

- ❖ **Dependências inexistentes**
- ❖ **Inputs sem produtor**
- ❖ **Ciclos no grafo de dependências**
- ❖ **Declaração duplicada de tarefas**

Todas as validações são testadas nos arquivos:

- **erros.pipe**
- **erro_ciclo.pipe**
- **erro_sintatico.pipe**

5.6 Tabela de Símbolos

Cada tarefa é registrada com:

- Nome
- Tipo (EXTRACT, TRANSFORM, LOAD)
- Inputs
- Outputs
- Dependências
- Linha de origem

A representação é usada pelas fases seguintes.

5.7 Geração de Código Intermediário (IR)

A IR gerada é composta por:

- **metadata**
- **artifacts**
- **tasks**
- **dependencies**
- **execution_order**

Exemplo real (trecho):

```
{  
  "id": "transform_data",  
  "type": "TRANSFORM",  
  "inputs": ["raw_data"],  
  "outputs": ["processed_data"],  
  "depends_on": ["extract_source"]  
}
```

A ordem de execução é calculada via ordenação topológica.

5.8 Simulador de Execução

O simulador:

- valida dependências
- registra artefatos produzidos
- simula EXTRACT, TRANSFORM, LOAD com delays
- apresenta logs com cores
- gera resumo final da execução

Exemplo de saída:

[1/3] Executando: extract_source (EXTRACT)

-> Artefato gerado: raw_data OK

5.9 Resultados dos Testes

Foram utilizados:

- **pipeline_simples.pipe**
- **pipeline_paralelo.pipe**
- **pipeline_complexo.pipe**
- **vendas_completo.pipe**

Todos foram compilados com sucesso e executados no simulador, validando:

- ordem de execução correta
- dependências satisfeitas
- ausência de ciclos
- produção e consumo adequado de artefatos

O compilador também identificou erros corretamente nos arquivos de teste negativo.

6. CONSIDERAÇÕES FINAIS

O projeto permitiu construir um compilador completo, modular e funcional para a linguagem PipeLang. A atividade integrou conceitos fundamentais de compiladores, como:

1. **análise léxica e sintática;**
2. **construção de AST;**
3. **tabela de símbolos;**
4. **análise semântica;**

5. geração de IR;

6. execução simulada.

O resultado final é um sistema capaz de interpretar e executar pipelines ETL simples e complexos, cumprindo integralmente os requisitos da atividade.

7. REFERÊNCIAS

Aho, A. V.; Lam, M.; Sethi, R.; Ullman, J. Compiladores: Princípios, Técnicas e Ferramentas. 2. ed. Pearson, 2008.

Sebesta, R. Conceitos de Linguagens de Programação. 10. ed. Bookman, 2013.

Documentação Python. Disponível em: <https://docs.python.org>

8. ANEXOS

```
pipeline vendas {  
  task extrair_dados {  
    from 'database_vendas'  
    to 'dados_brutos'  
  };  
  
  task filtrar_clientes {  
    from 'dados_brutos'  
    filter idade >= 18 and status == 'ativo' or vip == 'true'  
    to 'clientes_validos'  
    after extrair_dados  
  };  
  
  task calcular_metricas {  
    from 'clientes_validos'  
    map total_vendas = 'sum(vendas)'  
    to 'metricas'  
    after filtrar_clientes  
  };  
  
  task carregar_warehouse {  
    from 'metricas'  
    to 'data_warehouse'  
    after calcular_metricas  
  }  
}
```

Figura 1. Sintaxe pipelang

```

=====
COMPILADOR PIPELANG
=====
Arquivo: exemplos/vendas_completo.pipe

[1/5] Analise Lexica...
      61 tokens identificados
[2/5] Analise Sintatica...
      Pipeline 'vendas' com 4 tarefa(s)
[3/5] Conversao AST -> Tabela de Simbolos...
      4 tarefa(s) na tabela de simbolos
[4/5] Analise Semantica...
      Nenhum erro semantico encontrado
[5/5] Geracao deCodigo Intermediario...
      IR gerada com sucesso

=====
[OK] COMPILACAO CONCLUIDA COM SUCESSO!
=====

```

Figura 2. Sintaxe compilada

```

=====
== Tabela de Símbolos ==
- extrair_dados : EXTRACT
  outputs: dados_brutos
- filtrar_clientes : TRANSFORM
  inputs:  dados_brutos
  outputs: clientes_validos
  deps:   extrair_dados
- calcular_metricas : TRANSFORM
  inputs:  clientes_validos
  outputs: metricas
  deps:   filtrar_clientes
- carregar_warehouse : LOAD
  inputs:  metricas
  deps:   calcular_metricas
=====

```

Figura 3. Tabela de símbolos

```

=====
PSEUDO-CODIGO DO PIPELINE
=====
# Fonte: exemplos/vendas_completo.pipe
# Tarefas: 4

ORDEM DE EXECUCAO:
  1. extrair_dados
  2. filtrar_clientes
  3. calcular_metricas
  4. carregar_warehouse

TAREFAS:

TASK extrair_dados (EXTRACT):
  EXECUTE: extract_operation()
  WRITE: dados_brutos
  MARK_COMPLETE: extrair_dados

TASK filtrar_clientes (TRANSFORM):
  WAIT_FOR: extrair_dados
  READ: dados_brutos
  EXECUTE: transform_operation()
  WRITE: clientes_validos
  MARK_COMPLETE: filtrar_clientes

TASK calcular_metricas (TRANSFORM):
  WAIT_FOR: filtrar_clientes
  READ: clientes_validos
  EXECUTE: transform_operation()
  WRITE: metricas
  MARK_COMPLETE: calcular_metricas

TASK carregar_warehouse (LOAD):
  WAIT_FOR: calcular_metricas
  READ: metricas
  EXECUTE: load_operation()
  MARK_COMPLETE: carregar_warehouse
=====

```

Figura 4. Pseudo-código

```

=====
SIMULADOR DE PIPELINE - Pipelang v1.0
=====

Pipeline: <compiled>
Total de tarefas: 4
Compilado em: 2025-11-19T23:00:05.182206

[1/4] Executando: extrair_dados (EXTRACT)
-> Extraindo dados da fonte...
-> Artefato gerado: 'dados_brutos' OK
-> Tempo: 0.33s
-> Status: COMPLETO OK

[2/4] Executando: filtrar_clientes (TRANSFORM)
-> Dependência satisfeita: extrair_dados OK
-> Lendo artefato: 'dados_brutos' (de extrair_dados) OK
-> Transformando dados...
-> Artefato gerado: 'clientes_validos' OK
-> Tempo: 0.69s
-> Status: COMPLETO OK

[3/4] Executando: calcular_metricas (TRANSFORM)
-> Dependência satisfeita: filtrar_clientes OK
-> Lendo artefato: 'clientes_validos' (de filtrar_clientes) OK
-> Transformando dados...
-> Artefato gerado: 'metricas' OK
-> Tempo: 0.73s
-> Status: COMPLETO OK

[4/4] Executando: carregar_warehouse (LOAD)
-> Dependência satisfeita: calcular_metricas OK
-> Lendo artefato: 'metricas' (de calcular_metricas) OK
-> Carregando dados no destino final...
-> Dados carregados com sucesso OK
-> Tempo: 0.33s
-> Status: COMPLETO OK

=====
EXECUÇÃO CONCLUÍDA COM SUCESSO! OK

Tarefas executadas: 4/4
Tempo total: 2.09s

Tempo por tarefa:
  • extrair_dados: 0.33s (completed)
  • filtrar_clientes: 0.69s (completed)
  • calcular_metricas: 0.73s (completed)
  • carregar_warehouse: 0.33s (completed)
=====

```

Figura 5. Simulação completa